

Формально верифицированный статический анализатор байткода eBPF

Тимофей Черганов
cherganov@gmail.com

01.07.2026

extended Berkeley Packet Filter (eBPF)^{1 2} – это подсистема ядра Linux, позволяющая запускать программы в контексте ядра для расширения его функциональности без необходимости изменения исходного кода ядра или загрузки модулей.

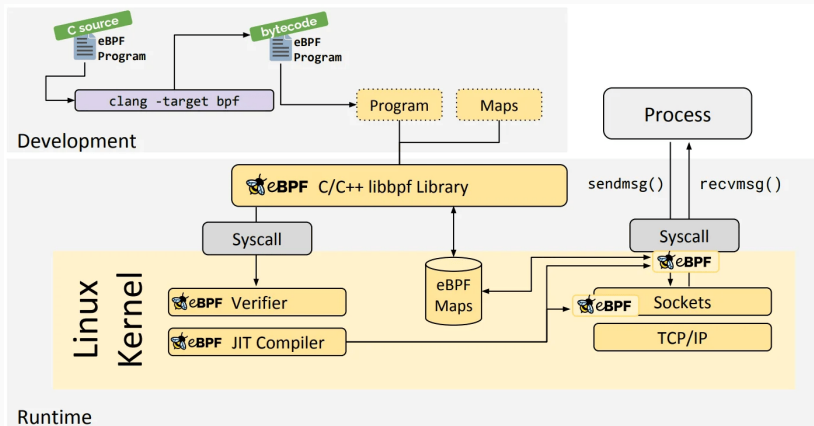
Используют для:

- фильтрации трафика и балансировки нагрузки;
- профилирования, трассировки и измерения производительности;
- наблюдения за событиями безопасности.

¹<https://ebpf.io>

²B. Gregg. [BPF: A New Type of Software](#), 2019

eBPF



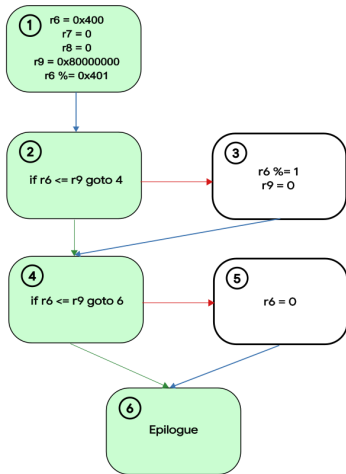
Запуск в контексте ядра предъявляет строгие требования к безопасности запускаемых программ, поэтому при загрузке в пространство ядра eBPF verifier^{3 4} проводит их анализ, который призван отклонить нежелательные программы.

eBPF verifier – большая программа со сложной логикой (~ 20 kloc). Ошибки в ней могут приводить к загрузке программ, которые могут использоваться злоумышленниками для компрометации системы: CVE-2020-8835, CVE-2021-3490, CVE-2021-4159, CVE-2022-23222, CVE-2023-2163, CVE-2024-41003.

³[eBPF verifier source code](#)

⁴[eBPF documentation](#)

eBPF Verifier: CVE-2023-2163



Возможные пути:

- 1:2:3:4:5:6
- 1:2:3:4:6
- 1:2:4:5:6
- 1:2:4:6

Верификатор анализирует только 1:2:3:4:5:6, ошибочно отсекая другие пути выполнения^a.

^a [A deep dive into CVE-2023-2163: How We found and Fixed an eBPF Linux Kernel Vulnerability](#)

Статические анализаторы пытаются автоматически установить некоторые свойства программ без их выполнения. Такие анализаторы можно использовать по-разному, например:

- для поиска ошибок, допускаемых программистами;
- в качестве верификаторов, гарантирующих соответствие программы определенным свойствам.

При верификации программ *корректность анализа* имеет первостепенное значение: если анализатор не выдает предупреждений, программа должна быть свободна от класса ошибок, отслеживаемых анализатором.

Статический анализ: пример

```
if x >= 0 then  
    y := x + 1  
else  
    y := 1  
x := x / y
```

Как убедиться, что любое выполнение программы не приводит к ошибке деления на 0?

Статический анализ: пример

```
{  $x = 10, y = 0$  }  
if  $x \geq 0$  then  
  {  $x = 10, y = 1$  }  
   $y := x + 1$   
  {  $x = 10, y = 11$  }  
else  
  { - }  
   $y := 1$   
  { - }  
{  $x = 10, y = 11$  }  
 $x := x / y$   
{  $x = 0, y = 11$  }
```

```
{  $x \in [-\infty, +\infty], y \in [-\infty, +\infty]$  }  
if  $x \geq 0$  then  
  {  $x \in [0, +\infty], y \in [-\infty, +\infty]$  }  
   $y := x + 1$   
  {  $x \in [0, +\infty], y \in [1, +\infty]$  }  
else  
  {  $x \in [-\infty, -1], y \in [-\infty, +\infty]$  }  
   $y := 1$   
  {  $x \in [-\infty, -1], y \in [1, 1]$  }  
  {  $x \in [-\infty, +\infty], y \in [1, +\infty]$  }  
 $x := x / y$   
{  $x \in [-\infty, +\infty], y \in [1, +\infty]$  }
```

Полученная аппроксимация является корректной, т.к. абстрактные значения содержат все возможные конкретные значения. Это позволяет утверждать, что выполнение операции не приводит к ошибке деления на 0 для любого конкретного выполнения программы.

Абстрактная интерпретация

Абстрактная интерпретация – это способ корректной аппроксимации семантики программ.

При аппроксимации:

- конкретные значения переменных заменяются абстрактными;
- конкретная семантика заменяется на семантику, работающую с этими абстрактными значениями.

Для этого *абстрактные значения* должны обладать:

- структурой решетки⁵;
- отображением конкретизации, связывающим абстрактные значения с конкретными;
- абстрактными операциями, соответствующими конкретным операциям языка программирования.

⁵ *Решеткой* называется частично упорядоченное множество, в котором любое конечное подмножество обладает точной верхней и точной нижней гранями.

Абстрактная интерпретация: семантика

$c \in com ::= \mathbf{skip} \mid x := e \mid c_1; c_2 \mid \mathbf{if } e \mathbf{ then } c_1 \mathbf{ else } c_2 \mid \mathbf{while } e \mathbf{ do } c$

$$\llbracket \mathbf{skip} \rrbracket(a) = a$$

$$\llbracket x := e \rrbracket(a) = a[x \mapsto \llbracket e \rrbracket(a)]$$

$$\llbracket c_1; c_2 \rrbracket(a) = \llbracket c_2 \rrbracket(\llbracket c_1 \rrbracket(a))$$

$$\llbracket \mathbf{if } e \mathbf{ then } c_1 \mathbf{ else } c_2 \rrbracket(a) = \llbracket c_1 \rrbracket(a) \vee \llbracket c_2 \rrbracket(a)$$

$$\llbracket \mathbf{while } e \mathbf{ do } c \rrbracket(a) = \text{postfixpoint of } (F(x) = a \vee \llbracket c \rrbracket(x))$$

Абстрактная интерпретация: корректность



$$\forall c \in com, a \in \Sigma^\sharp : \llbracket c \rrbracket (\gamma(a)) \subseteq \gamma(\{c\}(a))$$

Доказательство: индукцией по синтаксису команд (абстрактная семантика каждой команды определена корректно, если ее компоненты имеют корректную абстрактную семантику).

Абстрактная интерпретация байткода eBPF

- Выбрать формальную систему
- Формализовать синтаксис eBPF инструкций
- Реализовать парсинг байткода eBPF
- Формализовать семантику eBPF инструкций
- Определить абстрактную семантику
- Определить абстрактные домены
- Доказать корректность аппроксимации
- Реализовать вывод сообщений об ошибках

Примеры формальных систем: FOL + ZFC, TG, HOL, Martin-Löf type theory.

Rocq⁶ – это система для работы с формальными доказательствами. Она предоставляет язык для записи математических определений, алгоритмов и теорем, а также среду для интерактивного построения доказательств.

зависимые типы для утверждений, индуктивные типы для синтаксиса и семантики, классы типов для алгебраических структур, тактики для автоматизации доказательств, экстракция кода в OCaml

⁶<https://rocq-prover.org>

Синтаксис eBPF инструкций

```
insn ::= (* Arithmetic instructions *)
        | binop width op reg (reg | int)
        | neg width reg
        (* Byte swap instruction *)
        | end endianness reg swap_width
        (* Jump instructions *)
        | ja (int16 | int)
        | jmp width jmpop reg (reg | int) int16
        | call call_type int
        | exit
        (* Load and store instructions *)
        | ld reg immop int int
        | ldx type reg reg int16
        | st type reg (reg | int) int16
        | sta type atomicop reg reg int16

prog ::= insn prog | ε
```

BPF Instruction Set Architecture

<https://www.kernel.org/doc/html/latest/bpf/standardization/instruction-set.html>

Синтаксис eBPF инструкций

```
Inductive insn : Type :=
(* Arithmetic instructions *)
| BINOP (w : op_width) (op : binop) (dst : reg) (src : reg + int)
| NEG (w : op_width) (dst : reg)

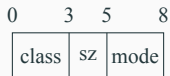
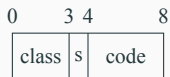
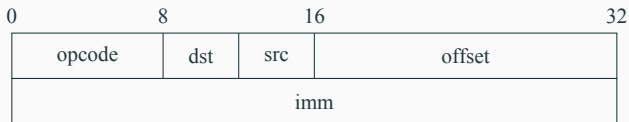
(* Byte swap instruction *)
| END (e : endianness) (dst : reg) (w : swap_width)

(* Jump instructions *)
| JA (off : int16 + int)
| JMP (w : op_width) (op : jmpop) (dst : reg) (src : reg + int) (off : int16)
| CALL (t : call_type) (imm : int)
| EXIT

(* Load and store instructions *)
| LD (dst : reg) (op : immop) (imm : int) (next_imm : int)
| LDX (t : type) (dst : reg) (src : reg) (off : int16)
| ST (t : type) (dst : reg) (src : reg + int) (off : int16)
| STA (t : type) (op : atomicop) (dst : reg) (src : reg) (off : int16).

Definition prog := list insn.
```

Парсинг байткода eBPF



Парсинг байткода eBPF

```
Definition parse_binop_src : Parser insn :=
  class <- parse_alu_class;
  s    <- X;
  code <- parse_binop;
  dst  <- parse_reg;
  src  <- parse_reg;
  code' <- parse_binop_off_src code class;
  imm  <- zeros imm_len;
  ret (BINOP class code' dst (inl src)).
```

```
Definition parse_arith : Parser insn :=
  parse_binop_src <|> parse_binop_imm <|> parse_neg.
```

```
Definition parse_insn : Parser insn :=
  (* Arithmetic and jump instructions *)
  parse_arith <|> parse_swap <|> parse_jump <|>

  (* Load and store instructions *)
  parse_regular_ldst <|> parse_ldx_memsx <|> parse_stx_atomic <|> parse_ld_imm.
```

```
Definition max_insn : nat := 1000000.
```

```
Definition parse : Parser prog :=
  many max_insn parse_insn.
```

EBinop

$$\frac{p[pc] = \mathbf{binop} \ w \ op \ dst \ src}{(pc, \text{regs}, \text{stack}) \xrightarrow{P} (pc + 1, \text{regs}[dst \mapsto \text{binop}(w, op, \text{regs}(dst), \text{val}(src))], \text{stack})}$$

EJumpTrue

$$\frac{p[pc] = \mathbf{jmp} \ w \ op \ dst \ src \ off \quad \text{jmp}(w, op, \text{regs}(dst), \text{val}(src)) = \text{true}}{(pc, \text{regs}, \text{stack}) \xrightarrow{P} (pc + 1 + off, \text{regs}, \text{stack})}$$

EJumpFalse

$$\frac{p[pc] = \mathbf{jmp} \ w \ op \ dst \ src \ off \quad \text{jmp}(w, op, \text{regs}(dst), \text{val}(src)) = \text{false}}{(pc, \text{regs}, \text{stack}) \xrightarrow{P} (pc + 1, \text{regs}, \text{stack})}$$

Семантика eBPF инструкций

```
Definition state : Type := Z * (reg -> int64) * (int64 -> int64).
```

```
Inductive eval_insn (p : prog) : state -> state -> Prop :=
```

```
| EBINOP : forall pc regs stack w op dst src,  
    insn_at p pc = Some (BINOP w op dst src) ->  
    eval_insn p (pc, regs, stack)  
    (pc + 1, update dst (eval_binop w op (regs dst) (value regs src)) regs, stack)  
...  
| EJMP : forall pc regs stack w op dst src off,  
    insn_at p pc = Some (JMP w op dst src off) ->  
    eval_insn p (pc, regs, stack)  
    (if (eval_jump w op (regs dst) (value regs src)) then  
        pc + 1 + Int16.signed off  
    else  
        pc + 1,  
        regs,  
        stack)  
...
```

Абстрактная семантика

```
20 Class Lattice (A : Type) := {  
21   le : relation A  
22   where "a  $\sqsubseteq$  b" := (le a b);  
23   join : A → A → A  
24   where "a  $\sqcup$  b" := (join a b);  
25   meet : A → A → A  
26   where "a  $\sqcap$  b" := (meet a b);  
27   bot : A  
28   where " $\perp$ " := bot;  
29   top : A  
30   where " $\top$ " := top;  
31  
32   le_refl      :  $\forall a$       : A, a  $\sqsubseteq$  a;  
33   le_trans     :  $\forall a\ b\ c$  : A, a  $\sqsubseteq$  b  $\rightarrow$  b  $\sqsubseteq$  c  $\rightarrow$  a  $\sqsubseteq$  c;  
34  
35   le_join_l    :  $\forall a\ b$    : A, a  $\sqsubseteq$  a  $\sqcup$  b;  
36   le_join_r    :  $\forall a\ b$    : A, b  $\sqsubseteq$  a  $\sqcup$  b;  
37   join_sup     :  $\forall a\ b\ c$  : A, a  $\sqsubseteq$  c  $\rightarrow$  b  $\sqsubseteq$  c  $\rightarrow$  a  $\sqcup$  b  $\sqsubseteq$  c;  
38  
39   meet_le_l    :  $\forall a\ b$    : A, a  $\sqcap$  b  $\sqsubseteq$  a;  
40   meet_le_r    :  $\forall a\ b$    : A, a  $\sqcap$  b  $\sqsubseteq$  b;  
41   meet_inf     :  $\forall a\ b\ c$  : A, c  $\sqsubseteq$  a  $\rightarrow$  c  $\sqsubseteq$  b  $\rightarrow$  c  $\sqsubseteq$  a  $\sqcap$  b;  
42  
43   bot_le       :  $\forall a$       : A,  $\perp$   $\sqsubseteq$  a;  
44   le_top       :  $\forall a$       : A, a  $\sqsubseteq$   $\top$ ;  
45 }.
```

Абстрактная семантика

```
364 Definition stepa (i : insn) (l : label) (ma : mema) : list2 (label * mema) :=
365   let ra := fst ma in
366   let ha := snd ma in
367   match i with
368   | ALU op dst src ⇒
369     let xa := ra dst in
370     let ya := eval_imm ra (imma src) in
371     let va := eval_alua op xa ya in
372     One (label_off l 1, (update dst va ra, ha))
373   | JMP j dst src off ⇒
374     let xa := ra dst in
375     let ya := eval_imm ra (imma src) in
376     let '(v1a, v2a) := eval_jmp_bwa j xa ya in
377     let '(u1a, u2a) := eval_jmp_bwa (not_jmp j) xa ya in
378     match src with
379     | inl src ⇒ Two
380       (label_off l off, (update src v2a (update dst v1a ra), ha))
381       (label_off l 1, (update src u2a (update dst u1a ra), ha))
382     | inr _ ⇒ Two
383       (label_off l off, (update dst v1a ra, ha))
384       (label_off l 1, (update dst u1a ra, ha))
385     end
386   | LD dst src off ⇒
387     let pa := addra ra src off in
388     let va := loada ha pa in
389     One (label_off l 1, (update dst va ra, ha))
390   | ST dst src off ⇒
391     let pa := addra ra dst off in
392     let xa := eval_imm ra (imma src) in
393     One (label_off l 1, (ra, storea ha pa xa))
394   end.
```

Абстрактный домен интервалов

```
14 Definition Interval := (Dual ZInf * ZInf)%type.
15
16 Definition y_Interval (x : Interval) :=  $\lambda$  (z : Z),
17   match x with ({|unDual := a|}, b)  $\Rightarrow$  a  $\sqsubseteq$  0fZ z  $\wedge$  0fZ z  $\sqsubseteq$  b end.
18
19 Instance y_Interval_monotone : MonotoneMap y_Interval.
20 Proof.
21   split; do 2 intros [[[]] []]; simpl; try easy; lia.
22 Qed.
23
24 Definition aconst : Z  $\rightarrow$  Interval :=
25    $\lambda$  z, (mkDual (0fZ z), 0fZ z).
26
27 Definition ZInf_add (x y : ZInf) : ZInf :=
28   match x, y with
29   | PInf, _
30   | _, PInf  $\Rightarrow$  PInf
31   | 0fZ m, 0fZ n  $\Rightarrow$  0fZ (m + n)%Z
32   | _, _  $\Rightarrow$  NInf
33   end.
34
35 Definition aplus : Interval  $\rightarrow$  Interval  $\rightarrow$  Interval :=
36    $\lambda$  '(a, b) '(c, d), (mkDual (ZInf_add (unDual a) (unDual c)), ZInf_add b d).
```

Корректность абстрактной интерпретации

```
874 Theorem sema_sound :  
875   ∀ (fuel : ℕ) (p : prog) (σa : statea),  
876     sem p (γ_state σa) ⊆ γ_state (sema fuel p σa).  
877 Proof.  
878   intros *.  
879   apply sem_is_lfp.  
880   _ apply map_monotone, sema_extensive.  
881   _ eapply le_trans, map_monotone, sema_is_fp.  
882   apply Stepa_sound.  
883 Qed.
```

Спасибо!